

Lab 02 — Questions

Answer without re-reading the theory. Always justify: a claim without a mechanism is worth nothing.

Question 1 [Understanding]

Write the **complete, explicit** name the engine builds from each of these spellings, explain the rule that tells whether the first part is a registry or a namespace — and say for each how Podman behaves with the short name:

```
nginx
bitnami/nginx
registry.mycompany.be:5000/base/nginx:1.25
```

Question 2 [Analysis]

Two servers, A and B, pulled `myapp/api:2.3` on the same day. Three weeks later, the team redeploys on B only, with the same command `podman pull myapp/api:2.3`. A bug appears on B and not on A. How is that possible when the version is "the same"? Which command proves it, and which practice would have avoided the problem?

Question 3 [Diagnosis]

A developer notices that `podman images` lists 40 images for a total of 62 GB in the `SIZE` column, while their WSL disk is only 100 GB and they never had a space problem. Do they really have 62 GB of images? Explain, give the command that shows the real figure, and say where those files physically live on their workstation.

Question 4 [Analysis]

A Dockerfile contains:

```
COPY credentials.json /tmp/credentials.json
RUN ./configure.sh && rm /tmp/credentials.json
```

The author claims the secret is not in the final image since they deleted it. Are they right? Explain precisely what the image contains, and why the `rm` changes nothing about the problem.

Question 5 [Understanding]

You push a second version of your Spring Boot image to the registry. It weighs 310 MB, the previous one weighed 308 MB. Yet the `push` only transfers 61 MB. Explain the mechanism, then say what you should change in your Dockerfile if the push transferred all 310 MB every time.

Question 6 [Diagnosis]

```
$ podman images
REPOSITORY          TAG          IMAGE ID          SIZE
localhost/api       2.0         f3a1b9c02d11     310 MB
localhost/api       1.9         f3a1b9c02d11     310 MB
<none>              <none>      8b2c74e91a03     295 MB
```

Comment on this output: how many distinct images do you really see? Where does the `<none>` line come from? What exactly happens if you type `podman rmi api:1.9`? And why `localhost/`?

Question 7 [Analysis]

Your colleague builds the back-end image on their MacBook M3 and pushes it to the registry. Deployment on the staging server fails with `exec /usr/bin/java: exec format error`. Diagnose, and give two ways to fix it — one to unblock right now, the other so the problem never happens again.

Question 8 [Understanding]

`podman save` and `podman export` both produce a `.tar` archive. In which situation is each the right choice? What exactly is lost if you use `export` to carry a Spring Boot image to an isolated site? And what does `--format oci-archive` change on a `save`?

Question 9 [Analysis]

After a `podman pull` of a 400 MB image, you run the same command again: it finishes in a few seconds with no download. What did the engine actually check — and why did it cost almost nothing on the network?

Question 10 [Diagnosis]

Your company's CI fails intermittently on `podman pull node:22-alpine`, with the message `toomanyrequests: You have reached your pull rate limit`. Nothing changed in the pipeline. Explain the cause, why it appears "intermittently", and the two classic answers in a company.

Question 11 [Diagnosis]

You start a local registry (`podman run -d -p 5000:5000 registry:2`), then:

```
$ podman push localhost:5000/base/demo:1.0
Error: ... pinging container registry localhost:5000: Get "https://localhost:5000/v2/":
http: server gave HTTP response to HTTPS client
```

Your colleague, on Docker, never saw that message with the same registry. Explain the difference in philosophy, give two ways to make the `push` go through — and say why one of the two must never reach a versioned configuration file.

Question 12 [Diagnosis]

A `podman rmi my-api:1.0` returns:

```
Error: image used by 4c2e9a1b7d33...: image is in use by a container: consider listing
external containers and force-removing image
```

Explain the situation, say why the engine refuses, give the **clean** way to resolve it — then say what `podman rmi -f` does and why it is a bad idea here.

Question 13 [Analysis]

`podman history` on an image shows several layers of size `0B` and one layer of `180MB`. What are the `0B` layers, and why do they exist at all? How does this reading concretely help you reduce the size of an image?

Question 14 [Understanding]

Your team hesitates between three tagging strategies for the back-end image: (a) `api:latest` overwritten at every build, (b) `api:1.4.2` following the application version, (c) `api:1.4.2-b318-a9f3c21` including the build number and the Git *commit*. Discuss the three from the point of view of **rollback in production** and **incident diagnosis**.